

Distilling Script Knowledge from Large Language Models for Constrained Language Planning

Mayukha Sista
FE67016



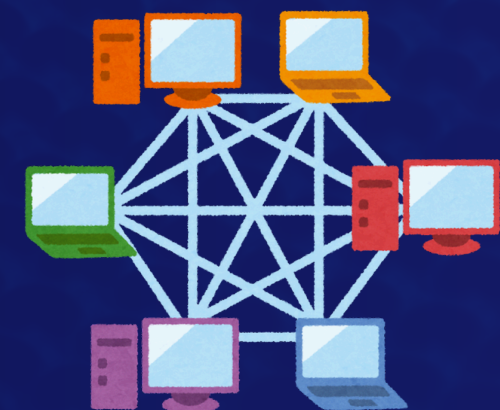
SUMMARY

This paper presents constrained language planning, in which language models produce detailed plans for certain objectives while following to constraints. The authors demonstrate that while large language models like InstructGPT are capable of producing reasonable and fluent plans for **abstract goals**, they frequently fall short of following to **specific constraints**.

Their tests reveal that CoScript-trained models significantly outperform constrained planning, proving that high-quality task-specific data can help smaller models perform on level with, or even better than, much bigger LLMs in certain evaluations.



In order to improve InstructGPT's planning accuracy from 69% to 95%, they suggest an **over-generate-then-filter** method that creates several candidate scripts and chooses the most constraint-faithful one. In order to train more specialised models, the authors employ this technique to generate **CoScript**, a dataset containing 55,000 constrained goals and matching scripts.



PROBLEM

For abstract goals like *"Make a cake,"* Large Language Models (LLMs) can provide efficient step-by-step plans, but they often fail when the goal has **specific constraints**. For instance, when instructed to *"Make a cake for diabetics,"* InstructGPT produced a recipe that appeared logical but still contained sugar, ignoring the diabetic limitation. This shows that the generated plan may not always be faithful to the user's requirements even if the instructions are clear and coherent.



MOTIVATION

Previous research on language planning mostly concentrated on **generic tasks**, with **little attention** paid to planning for specific goals with constraints. Requirements relating to **what should be done**, **how** it should be done, or **why/for whom** it is being done are often present in real-world jobs.

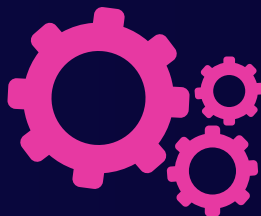
Script for the Goal:
Make a Cake for Diabetics

- 1) Gather your ingredients.
- 2) Preheat oven to 350 °F. Grease an 8-inch (20 cm) cake pan with butter or margarine.
- 3) In a medium bowl, combine flour, baking powder and salt.
- 4) In a large bowl, cream butter or margarine and **sugar** together until light and fluffy. Beat in eggs, one at a time.
- 5) Add vanilla extract and almond extract.
- 6) Gradually add flour mixture to the large bowl, mixing until just blended.
- 7) Pour batter into prepared cake pan.
- 8) Bake cake for 30 minutes.
- 9) Remove cake from oven and let cool before serving.

Sugar for diabetics?!

InstructGPT
(text-davinci-002)

Constrained Language Planning



The task of putting together a detailed plan for a specific goal while making sure that the steps generated faithfully satisfy its constraints is known as Constrained Language Planning.

Constraint Type 1: Modifier
Definition: A word, an adjective or a phrase that modifies or constrains an abstract goal.
Ex.1: Make a chocolate cake.
Ex.2: Make a pink cake.
Constraint Type 2: Method
Definition: A tool or specified mode that controls the process for achieving the goal.
Ex.1: Make a cake with an oven .
Ex.2: Make a cake by using cake mix .
Constraint Type 3: Intent
Definition: An additional purpose or demand when completing the goal.
Ex.1: Make a cake for wedding .
Ex.2: Make a cake for diabetics .

Three categories of constraints are defined by the authors: "**Make a chocolate cake**" is an example of a **Modifier**, "**Make a cake with an oven**" is an example of a **Method** and **Intent**, which adds a goal or condition, like "**Make a cake for a wedding**"



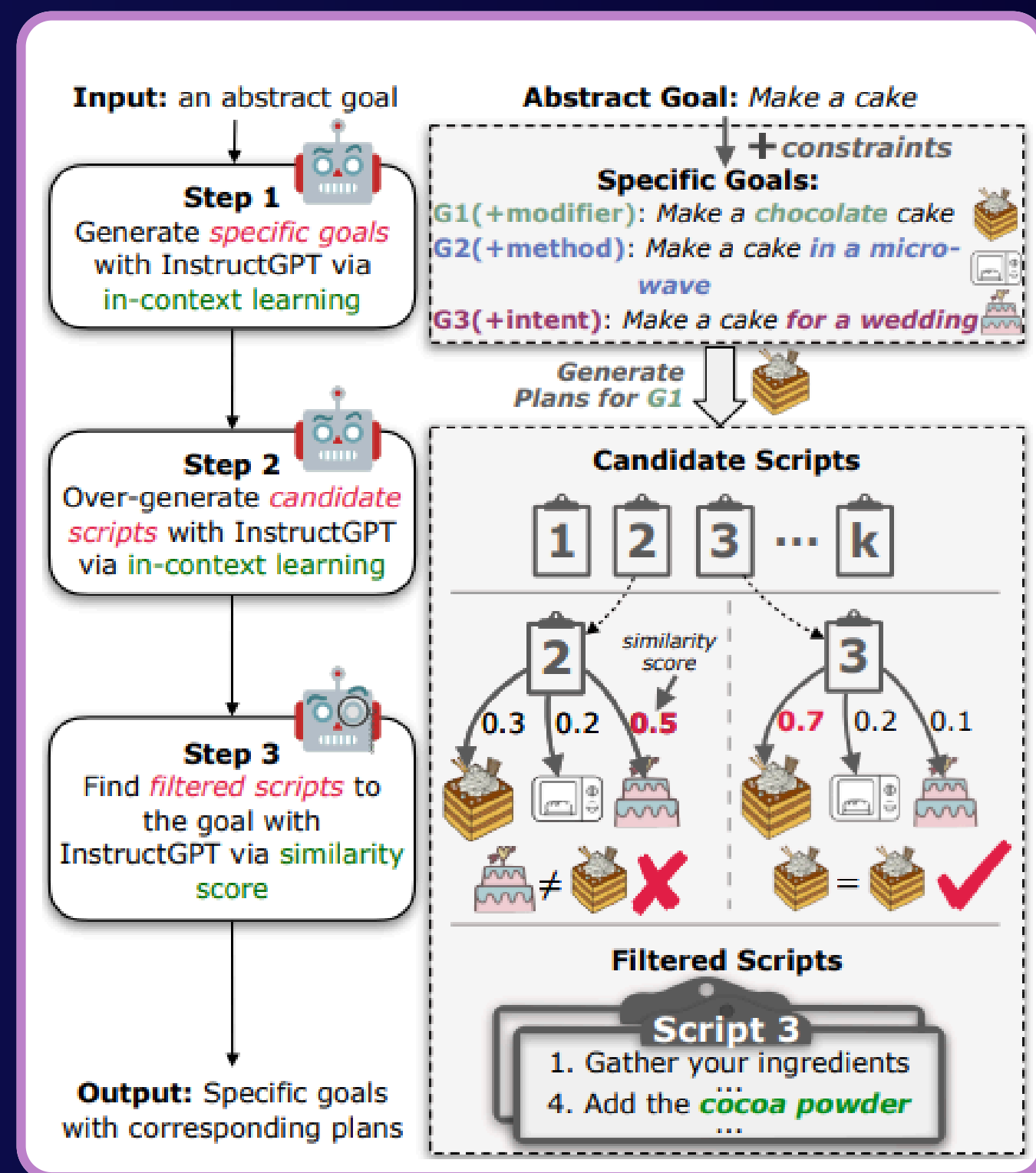
PROPOSED METHOD

Over-Generate → Filter

For any particular goal, InstructGPT initially creates **several candidate scripts** rather than producing a single script and accepting it straight away.

Semantic similarity between each created script and the particular goal is then used to evaluate these candidates. The authors calculate cosine similarity to assess how well each script aligns with the desired goal. **Scripts that specifically include the goal limitation are also rewarded.**

When the target specific goal exceeds other goals generated from the same abstract goal on the basis of similarity score, the **script is preserved**.



COSCRIPT DATASET

The authors create CoScript, a huge data set created specifically for constrained language planning, using their over-generate-then-filter model. In order to give InstructGPT samples, they start with 14,945 wikiHow article titles as abstract seed goals and obtain 34,260 related wikiHow goals and scripts. Three steps are then taken to create the dataset:

- (1) InstructGPT creates specific goals by adding modifier, method, or intent constraints to the abstract goals
- (2) it over-generates multiple candidate scripts for each specific goal
- (3) the proposed filtering method selects the scripts that are nearest to their constraints.

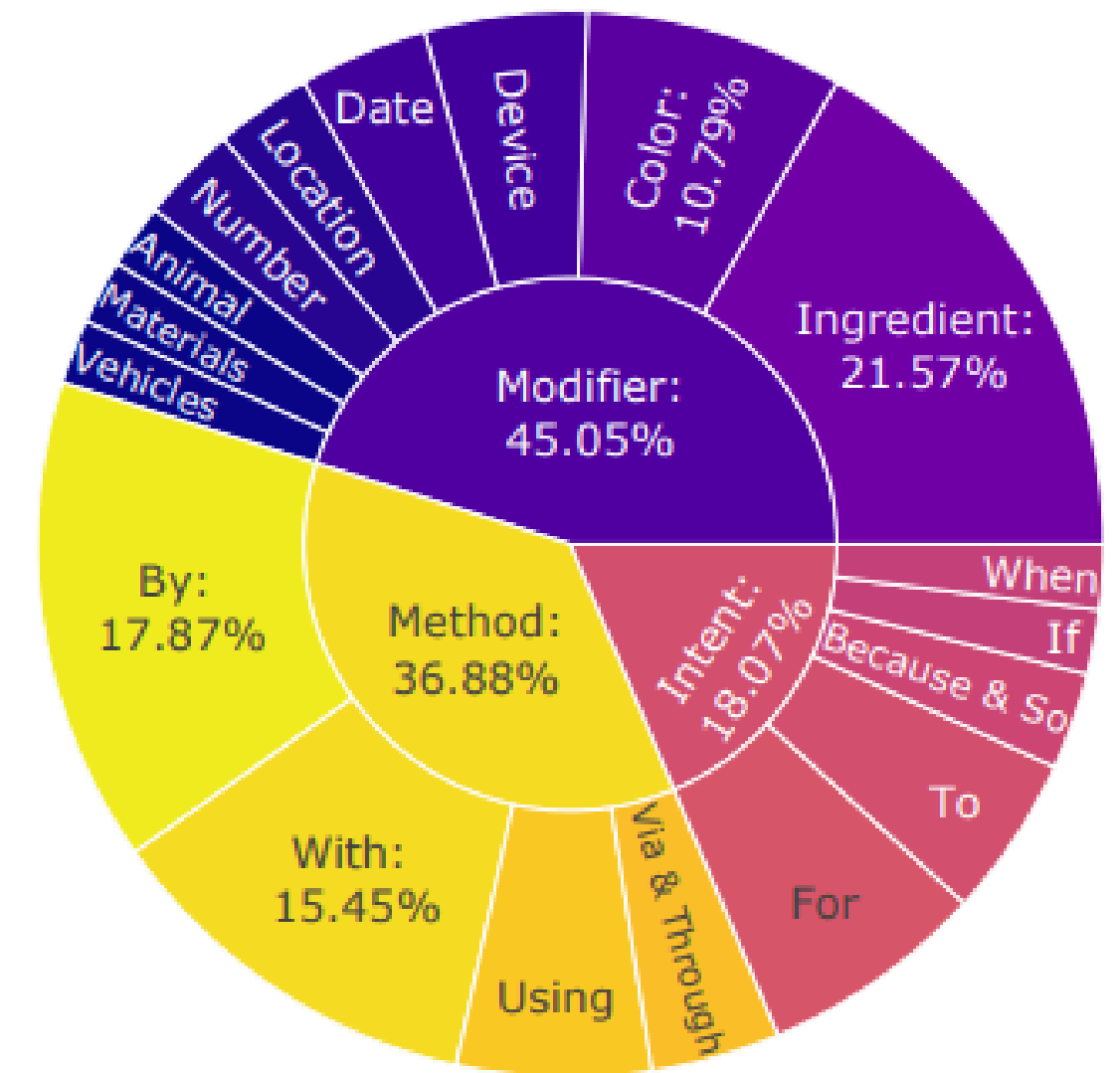


Figure 5: Statistics of constraint types in CoScript dataset, with representative topic categories or the first words for each constraint type.

EXPERIMENTS & MAIN RESULTS

The authors assess if LLMs can produce plans that accurately adhere to intent, method, and modifier constraints. InstructGPT achieves **97% accuracy** on general abstract goals, but **when particular limitations are added**, its performance **drastically declines**. Simply adding "Let's think step by step" does not enhance the 175B InstructGPT's overall accuracy on constrained planning, which is only **69%**. On the other hand, the suggested **over-generate-then-filter** approach improves total accuracy by **26 percentage points**, reaching **95%**. The biggest obstacle is intent limitations, where the suggested approach increases accuracy to **92.86%** whereas typical InstructGPT only achieves **42.86%**.

These results show that maintaining the generated steps faithfulness to the given constraint, instead of generating fluent plans, is the main challenge.

Model	Modifier	Method	Intent	All
Retrieval	26.67	38.89	35.71	34.00
T0 (11B)	30.00	21.12	25.00	24.00
Flan-T5 (11B)	50.00	42.25	31.25	42.00
GPT-3 (1.3B)	13.33	12.96	18.75	14.00
GPT-3 (6.7B)	23.33	7.40	25.00	15.00
GPT-3 (175B)	30.00	22.22	25.00	25.00
-----	-----	-----	-----	-----
Codex (175B)	46.67	55.56	18.75	47.00
-----	-----	-----	-----	-----
InstructGPT (1.3B)	20.00	22.22	28.57	22.00
InstructGPT (6.7B)	60.00	42.25	43.75	47.00
InstructGPT (175B)	73.33	74.08	42.86	69.00
+ "let's think step..."	70.00	75.92	50.00	68.00
-----	-----	-----	-----	-----
+ Our Method	96.67	98.15	92.86	95.00
w/ f_{sim} = SBERT	86.66	74.89	81.25	78.00
w/ f_{sim} = SimCSE	73.33	78.73	75.00	75.00
w/ f_{sim} = None	93.33	94.44	87.50	93.00

Related work

Paper	Focus	How this paper differs	
Sakaguchi et al. (2021) – “proScript: Partially Ordered Scripts Generation”	Goal-oriented script generation	CoScript handles constrained specific goals and is much larger	LINK
Huang et al. (2022) – “Language Models as Zero-Shot Planners	Uses LLMs for planning	This paper specifically addresses constraint faithfulness	LINK
West et al. (2022) – “Symbolic Knowledge Distillation: from General Language Models to Commonsense Models”	Distills knowledge from large LMs	This paper applies distillation specifically to constrained language planning	LINK



Strengths

01

Introduces a new problem

Constrained Language Planning, moving beyond generic goals toward specific real-world requirements



02

Simple but effective method

Over-generate-then-filter improves InstructGPT from 69% to 95% accuracy without requiring fine-tuning of the large model.



03

Large reusable dataset

CoScript provides 55,000 constrained goal-script pairs, substantially larger than proScript's 6,414 scripts



04

Effective knowledge transfer

T5-3B + retrieval reaches 71% manual accuracy, compared with 69% for the evaluated 175B InstructGPT setup.





Weakness

01

Single Constraint Only

Does not handle combinations of multiple constraints.

02

LLM-Generated Dataset Bias

CoScript may inherit InstructGPT's biases and generation patterns

03

Data Quality

~5% of training samples may still be incorrect.

04

Post-hoc Filtering

Selects better outputs rather than improving the LLM itself.



FUTURE WORK



Multi-
Constraint
Planning



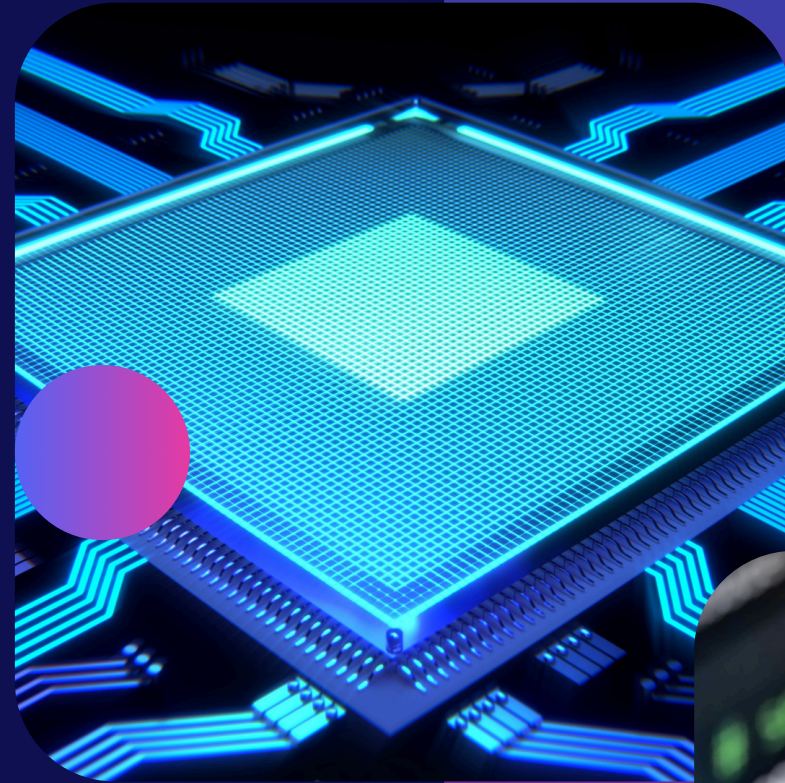
Improve the
Planner Itself



Improve
Dataset
Quality &
Diversity



Real-World &
Robotic
Planning





Thank
You!

